CS315 · ALGORITHMS ANALYSIS & DESIGN

Practice 1 Analysis & Sorting Solutions

Practice on Lecture 1 and Lecture 2 · 11 questions

Muharib

01 Solution Q1: arraySum

Line	# operations	Why
`sum ← 0`	1	An assignment.
`for i ← 0 to n − 1 do`	1 + (n + 1)	One initialization, and `n + 1` tests because the loop runs `n` times.
`sum ← sum + A[i]`	3n	Indexing, an addition and an assignment.
`{ increment counter i }`	2n	An addition and an assignment.
`return sum`	1	The return.
Total	**6n + 4**	O(n)

Watch out

Here the loop starts at 0, so it runs `n` times. In **arrayMax** it starts at 1 and runs `n − 1` times. Always count the iterations first.

02 Solution Q2: arrayMax with Temp

Line	Worst	Best
`currentMax ← A[0]`	2	2
`Temp ← A[0]`	2	2
`for i ← 1 to n − 1 do`	1 + n	1 + n
`if A[i] > currentMax then`	2(n − 1)	2(n − 1)
`currentMax ← A[i]`	2(n − 1)	0
`{ increment counter i }`	2(n − 1)	2(n − 1)
`return currentMax`	1	1
Total	**7n**	**5n + 2**

```
Worst:   2 + 2 + (1 + n) + 6(n − 1) + 1  =  7n
```

```
Best:    2 + 2 + (1 + n) + 4(n − 1) + 1  =  5n + 2
```

This is the same question as the 451 and 442 midterms. The expected answer there is `7n`, which is the lecture's `7n − 2` plus the two operations of the Temp line.

03 Solution Q3: nested loops

Line	# operations	Why
`count ← 0`	`1`	
`for i ← 0 to n − 1 do`	`n + 2`	`1 + (n + 1)`.
`for j ← 0 to n − 1 do`	`n(n + 2)`	The whole line runs `n` times, and each run costs `n + 2`.
`count ← count + 1`	$2n^2$	Two operations, `n × n` times.
`{ increment counter j }`	$2n^2$	
`{ increment counter i }`	$2n$	
`return count`	`1`	
Total	$\mathbf{5n^2 + 5n + 4}$	$O(n^2)$

$$1 + (n + 2) + (n^2 + 2n) + 2n^2 + 2n^2 + 2n + 1 \;=\; 5n^2 + 5n + 4$$

04 Solution Q4: the Big-Oh definition

a $5n + 3$ **is** $O(n)$

$$5n + 3 \le cn$$

$$(c - 5)n \ge 3$$

Pick $c = 6$: $n \ge 3$ → $c = 6$, $n_0 = 3$

b $4n^2 + 3n + 2$ is $O(n^2)$

$$4n^2 + 3n + 2 \leq 4n^2 + 3n^2 + 2n^2 = 9n^2 \quad \text{for } n \geq 1$$

$$c = 9, \quad n_0 = 1$$

c $3 \log n + 5$ is $O(\log n)$

For $n \geq 2$: $\log n \geq 1$ so $5 \leq 5 \log n$

$$3 \log n + 5 \leq 3 \log n + 5 \log n = 8 \log n$$

$$c = 8, \quad n_0 = 2$$

d n^3 is not $O(n^2)$

$$n^3 \leq cn^2$$

$$n \leq c$$

This cannot hold for all $n \geq n_0$, because c is a constant and n grows without bound.

Any other values that satisfy the inequality are also correct. In (a), for instance, $c = 8, n_0 = 1$ works.

05 Solutions Q5 and Q6

$$\text{Q5:} \quad 100 < \log n < 5n < n \log n < 4n^2 < n^3 < 2^n$$

#	f(n)	Big-Oh	Reason
a	$10^5n^2 + 10^8n$	$O(n^2)$	Drop the constants and the lower-order term.
b	$n(n + 1) / 2$	$O(n^2)$	It equals $n^2/2 + n/2$.
c	$3 \log n + \log \log n$	$O(\log n)$	$\log \log n$ is smaller than $\log n$.
d	$7n - 2$	$O(n)$	A polynomial of degree 1.
e	$2^n + n^3$	$O(2^n)$	The exponential grows faster than any polynomial.
f	500	$O(1)$	A constant.

06 Solution Q7: True / False

#	Answer	Reason
1	**F**	The same hardware and software **and the same inputs** must be used.
2	**T**	The slide's own words.
3	**T**	We count the maximum number of operations, which is the worst case.
4	**F**	It changes `T(n)` by a constant factor only.
5	**F**	$n^2 \leq cn$ gives $n \leq c$, which is impossible.
6	**F**	In all the cases.
7	**T**	When the numbers are already sorted.
8	**F**	That describes merge sort.
9	**T**	The definition of the RAM model.
10	**F**	Ω means greater than or equal. Less than or equal is O.

Statements 1, 2 and 3 appeared word for word in the midterms. I have no official answer key. The answers come from the slides.

07 Solution Q8: prefix averages

```
a)   A[0] = 4 / 1 = 4
     A[1] = (4 + 8) / 2 = 6
     A[2] = (4 + 8 + 6) / 3 = 6
     A[3] = (4 + 8 + 6 + 2) / 4 = 5          A = [4, 6, 6, 5]
```

b) Because it recomputes the sum from the start for every `i`. The inner loop runs `1 + 2 + … + (n − 1)` times. That is an arithmetic progression equal to `n(n − 1)/2`, so the time is `O(n²)`.

```
c)  Algorithm prefixAverages2(X, n)
        A ← new array of n integers
        s ← 0
        for i ← 0 to n − 1 do
            s ← s + X[i]
            A[i] ← s / (i + 1)
        return A
```

08 Solution Q9: insertion sort

i	key	Array after the iteration
–	–	5 2 4 6 1 3
2	2	2 5 4 6 1 3
3	4	2 4 5 6 1 3
4	6	2 4 5 6 1 3
5	1	1 2 4 5 6 3
6	3	1 2 3 4 5 6

b) The best case is an already sorted array: `O(n)`. The worst case is a reverse-sorted array: `O(n²)`.

c) Initialization, then maintenance, then termination.

09 Solution Q10: merge sort

Divide	[7 2 9 4 3 8 6 1]			
	[7 2 9 4]		[3 8 6 1]	
	[7 2]	[9 4]	[3 8]	[6 1]
	[7] [2]	[9] [4]	[3] [8]	[6] [1]
Merge	[2 7]	[4 9]	[3 8]	[1 6]
	[2 4 7 9]		[1 3 6 8]	
	[1 2 3 4 6 7 8 9]			

```
b)   T(n) = T(n/2) + T(n/2) + T(MERGE)
```

```
c)   Levels      =  log(16) + 1  =  4 + 1  =  5        (levels 0 to 4)
```

```
     Level 3     =  2³ = 8 problems,  each of size 16 / 2³ = 2
```

```
     Total work  =  6n(log(n) + 1)  =  6 × 16 × 5  =  480
```

10 Solution Q11: divide and conquer

a) Divide: break the problem into sub-problems of the same form. Conquer: solve them recursively. Combine: merge their solutions.

	Merge sort	Quick sort
Splitting	In the middle, two equal halves.	Around a pivot, possibly unequal.
Auxiliary array	Required.	Not required.
Merge step	Required.	Not required.
Worst case	$O(n \log n)$	$O(n^2)$